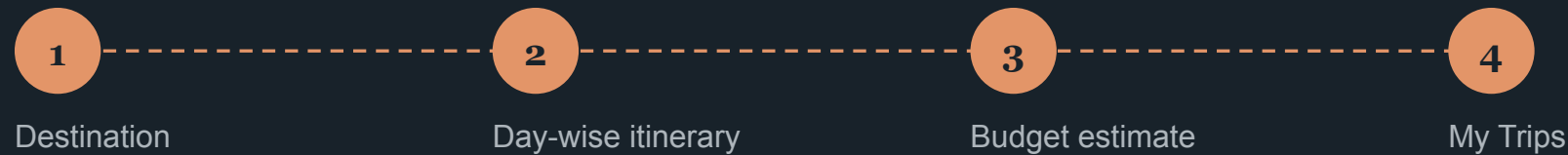


Smart Trip Planner

A web application that turns a destination, travel dates and a budget tier into a day-wise itinerary and a cost estimate.



PRESENTED BY

Udit Mishra

[[ENROLLMENT NO]]

Backend / Database / API

Hardik Vaghela

[[ENROLLMENT NO]]

Frontend / UI

Vraj [[SURNAME]]

[[ENROLLMENT NO]]

Documentation / UML / System Design

GUIDED BY

[[GUIDE NAME]]

TEAM

[[TEAM NO]]

PRESENTATION - 1

21 September 2026

Presentation Agenda

01

Study

Existing System

Need for the New System

Objective of the New System

Problem Definition

Scope of the Project

02

Proposed system

Core Components

Development Tools & Technologies

Assumptions and Constraints

03

Requirements

Functional Requirements

Targeted Users

04

Design

Use Case Diagram

Activity Diagram

Interaction Diagram

Class Diagram

Data Dictionary

05

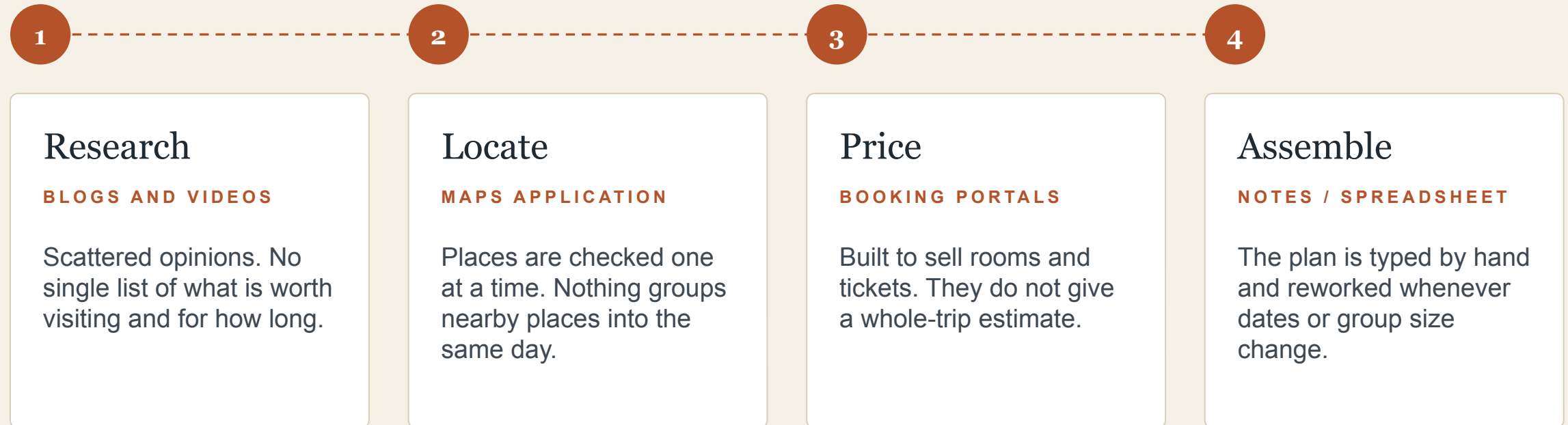
Progress

Implementation Status

Demo Flow

Questions

Existing System



Result

Hours of effort · no distance-aware day plan · budget is a guess · nothing saved in one place

Need for the New System

Information

EXISTING WAY

Spread over blogs, maps and portals

Day plan

Made by hand, distance ignored

Budget

A rough guess, often only the hotel

Changes

Rework the whole plan

Record

Screenshots and loose notes

SMART TRIP PLANNER

One curated catalogue: destinations with activities, fees, timings

Generated day-wise, stops ordered to cut travel

Stay, food, transport and activities; total and per person

Change an input and regenerate

Saved in My Trips with notes and status

Objective of the New System

01

Plan in one step

One form in, a complete day-wise itinerary out.

02

Less time on the road

Each day groups nearby places; stops are ordered by distance.

03

A budget you can read

Stay, food, transport and activities, with total and per-person cost in INR.

04

Try before sign-up

Anyone can preview a plan. Login is needed only to save it.

05

Works without the internet

Seeded data and a local distance provider by default; OpenStreetMap and Google are optional.

06

Secure and tested

JWT authentication, validated input, trips visible only to their owner, automated tests.

Problem Definition

Given a destination, travel dates, group size, budget tier, interests and pace, **produce a feasible day-wise itinerary that keeps travel between stops low, together with a transparent cost estimate — in one step, and keep it for later.**

Feasible = visit time + travel time fits the day: 6 h relaxed, 8 h balanced, 10 h packed.

THE TRAVELLER GIVES

- Destination
- Start and end date
- Travellers (1–12)
- Budget tier
- Interests
- Pace



THE SYSTEM RETURNS

Day-wise itinerary with start and end times
Budget: stay · food · transport · activities

Scope of the Project

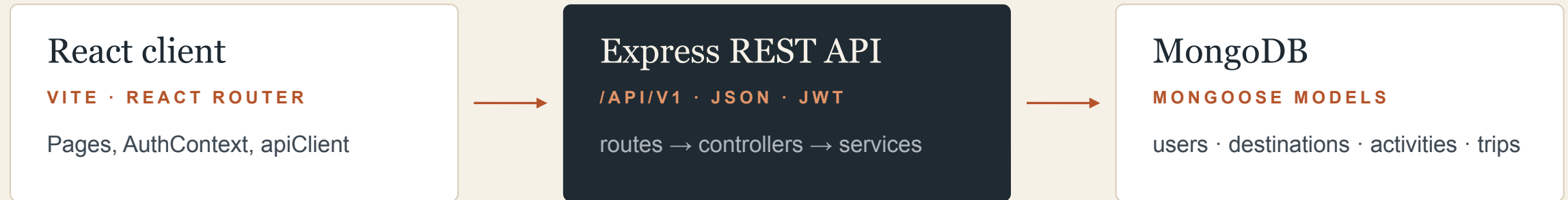
In scope

- 12 Indian destinations, 8–10 curated activities each
- Destination search, category filter and details page
- Day-wise itinerary from dates, interests and pace
- Budget estimate in three tiers: economy, standard, luxury
- Register, login, and My Trips: save, notes, status, delete
- Discover nearby places through the Places provider

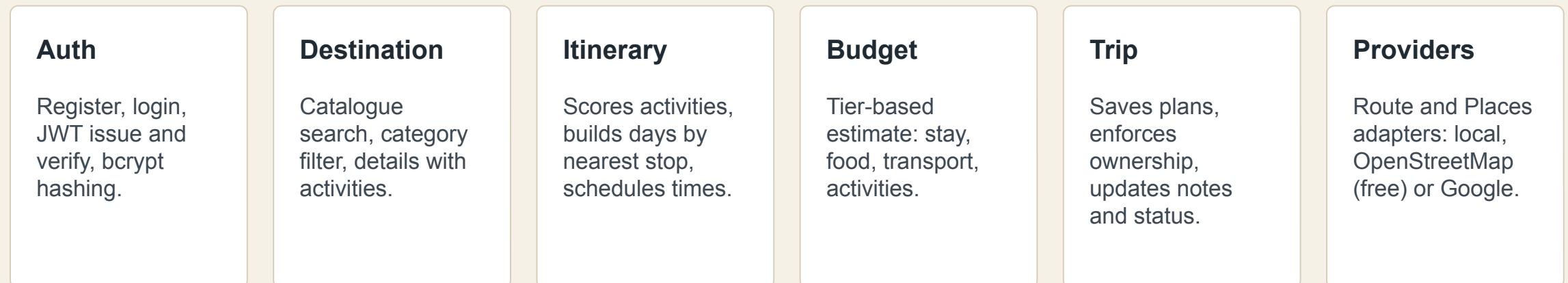
Out of scope

- Online payments
- Hotel, ticket or cab booking
- Chatbot or AI assistant
- Native mobile application
- Live prices and availability
- Admin screens (role field is reserved)

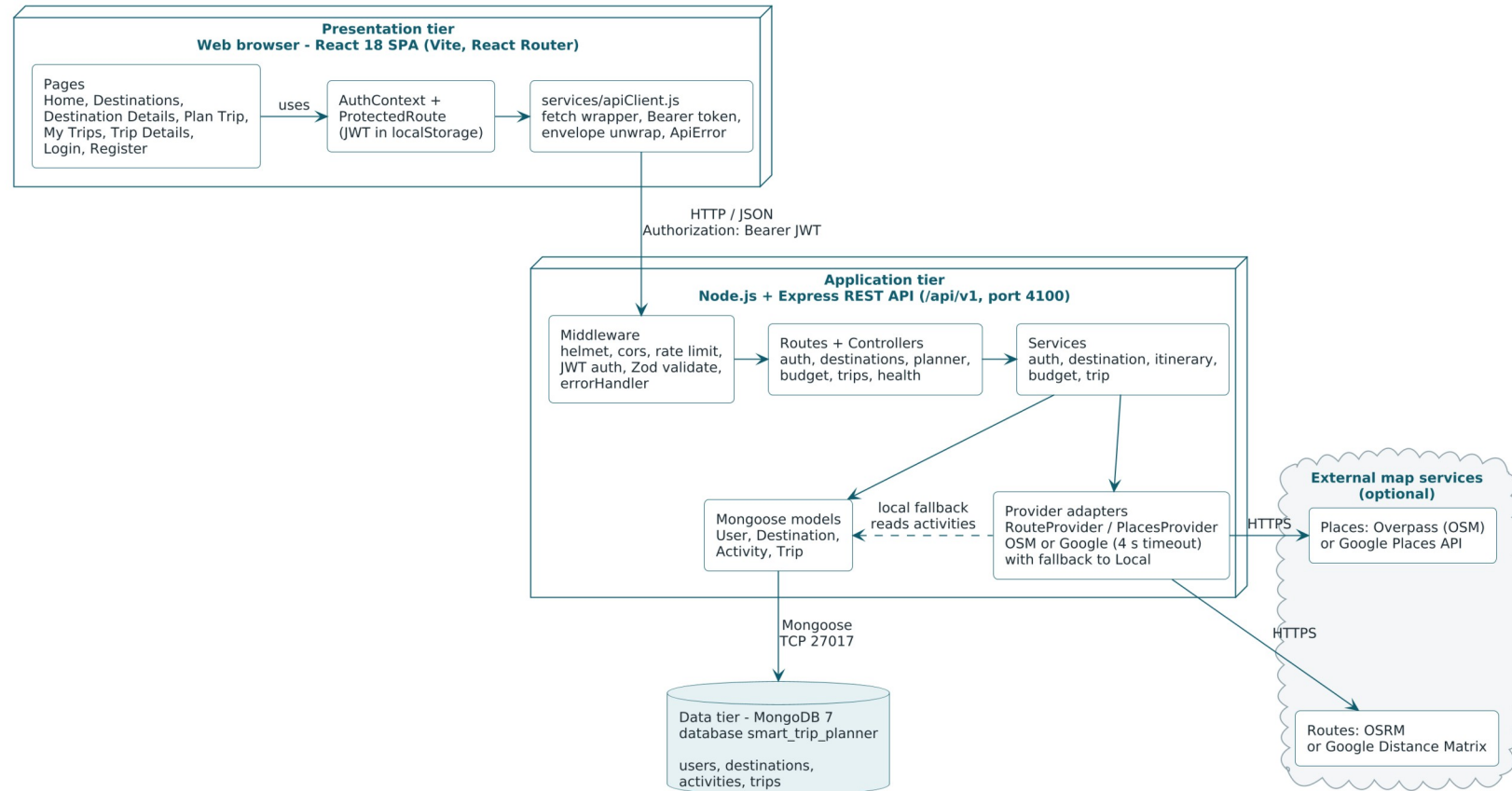
Core Components



SERVICES INSIDE THE API



Core Components — System Architecture



Browser → REST API → MongoDB. External map services are optional; every call falls back to the local provider.

Core Components — How the Plan Is Computed

Itinerary

- 1 Score** rating + 1.5 when the type matches an interest
- 2 Capacity** hours per day: relaxed 6 · balanced 8 · packed 10
- 3 Build the day** start at the best activity, keep adding the nearest one that still fits
- 4 Order** morning places first, evening places last
- 5 Schedule** from 09:00, travel time between stops, 60-minute lunch after 13:00

Budget

```
nights      = max(days - 1, 1)
rooms       = ceil(travellers ÷ 2)
stay        = rate.stay × rooms × nights
food        = rate.food × travellers × days
transport   = rate.transport × travellers × days
activities  = entry fees × travellers
total       = sum of the four
perPerson   = round(total ÷ travellers)
```

4 days, 2 travellers → **3 nights, 1 room.** rate = the destination's daily cost for the chosen tier.

Development Tools & Technologies

Layer	Technology	Role in the project
Frontend	React 18, Vite, React Router	Single-page client, routing, protected pages
Styling	HTML5, hand-written CSS3, design tokens	Responsive from 360 px, accessible forms, no UI kit
Backend	Node.js 20+, Express.js (ES modules)	REST API under /api/v1 with a uniform response envelope
Security	JWT, bcrypt, Zod, helmet, cors, rate limit	Login tokens, password hashing, input validation, hardening
Database	MongoDB 7, Mongoose	Four collections; itinerary and budget embedded in a trip
External APIs	Overpass (OpenStreetMap), OSRM; optional Google Places, Distance Matrix	Place discovery and road distance; local fallback
Testing	Vitest, Supertest, mongodb-memory-server	Unit tests for services, API tests on an in-memory database
Tooling	Git, Docker Compose, npm	Version control, one-command database, scripts and seed

Assumptions and Constraints

Assumptions

- Two travellers share one room
- Nights = days – 1, never less than one
- Costs are daily averages per tier, in whole INR
- Local road distance = straight line × 1.3 at 25 km/h
- Sightseeing starts at 09:00 with a one-hour lunch
- Figures are estimates, not quotations

Constraints

12

seeded Indian destinations

1–14

days per trip

1–12

travellers per trip

4 S

External API timeout, then local fallback

20 / 15 min

auth requests allowed per IP

Node 20+

and MongoDB 7 to run the system

Functional Requirements (1 of 2)

ID	The system shall...	Module
FR-01	register a user with name, email and password (min. 8 characters); reject a duplicate email	Auth
FR-02	log a user in with email and password and issue a JWT	Auth
FR-03	restore the session on reload; send a protected page to login and return afterwards	Auth
FR-04	list destinations with keyword search, category filter and pagination	Destinations
FR-05	show destination details: overview, best season, daily cost table, activities	Destinations
FR-06	discover nearby places for a destination, optionally by type	Destinations · Providers
FR-07	accept a plan: destination, dates, travellers, budget tier, interests, pace	Planner
FR-08	validate a plan: end $\geq$ start, at most 14 days, start not in the past, 1–12 travellers	Planner

Functional Requirements (2 of 2)

ID	The system shall...	Module
FR-09	generate a day-wise itinerary with timings, ordered to reduce travel between stops	Itinerary
FR-10	estimate the budget: stay, food, transport, activities, total and per person	Budget
FR-11	let a guest preview the generated plan without an account	Planner
FR-12	save a plan to My Trips after login; keep the pending plan across the login step	Trips
FR-13	recompute itinerary and budget on the server when saving; never trust client totals	Trips
FR-14	list the user's saved trips, newest first, with a status filter	Trips
FR-15	show trip details and allow editing of title, notes and status	Trips
FR-16	delete a trip; one user can never read or change another user's trip	Trips

Targeted Users

Student group

ECONOMY TIER · PACKED PACE

NEED Needs the per-person cost before committing.

GETS Budget split per person; up to 12 travellers.

Family on holiday

STANDARD TIER · RELAXED PACE

NEED Wants short days and little road time.

GETS 6-hour days, nearby stops grouped together.

First-time visitor

ANY TIER · BALANCED PACE

NEED Does not know what to see or how long it takes.

GETS Curated activities with duration, fee and best time.

ROLE 1

Guest

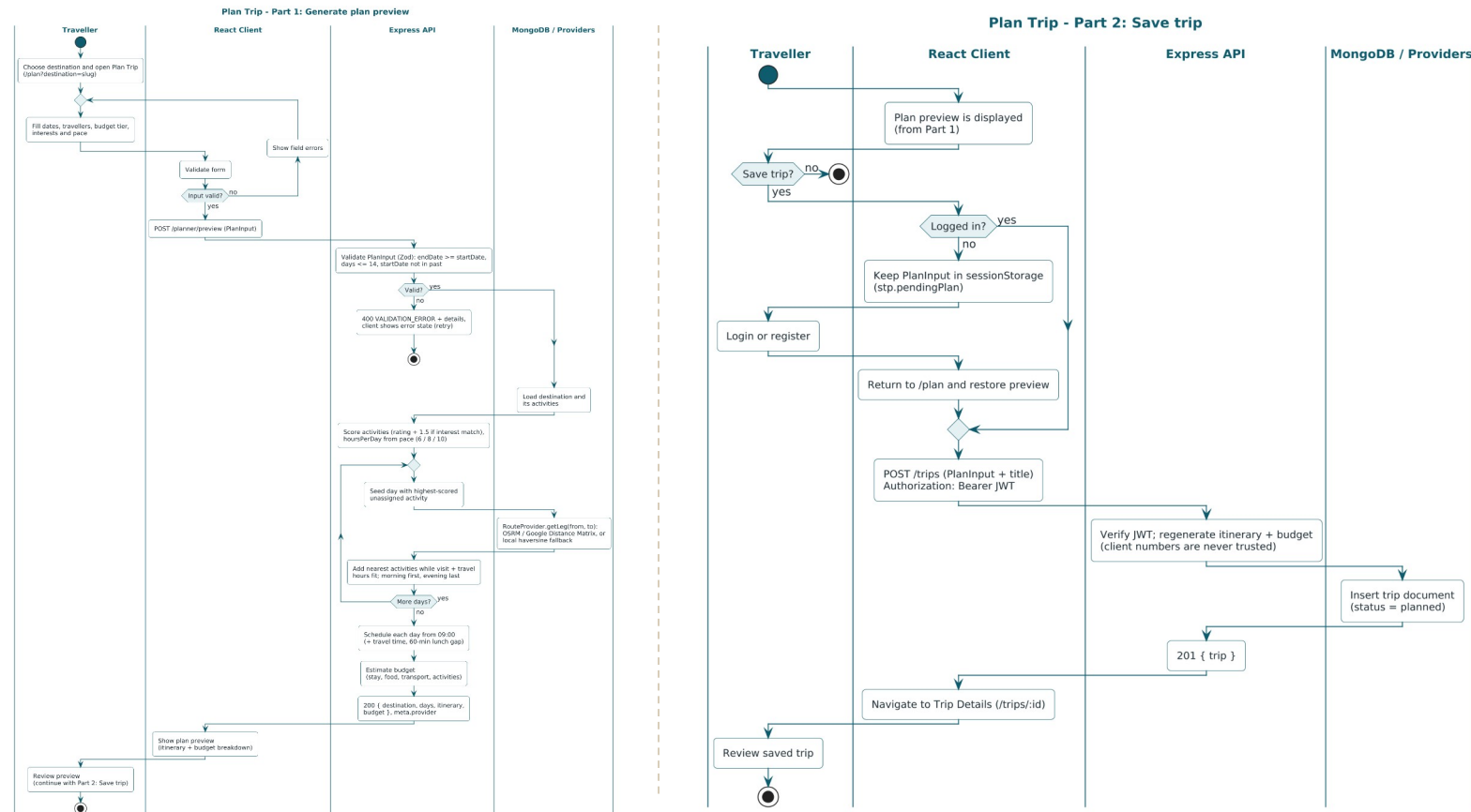
No account. Browses destinations, opens details, generates and previews a plan.

ROLE 2

Registered traveller

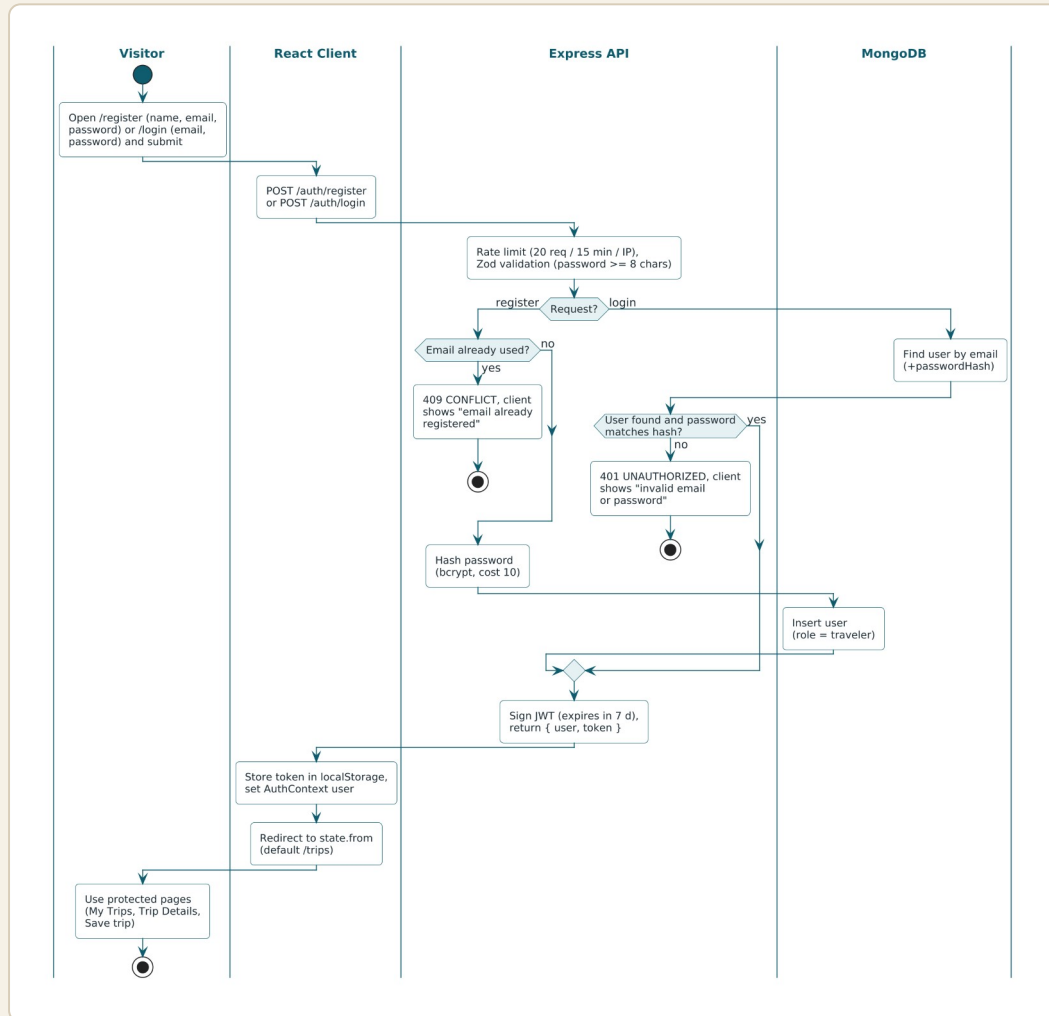
Logged in. Everything a guest can do, plus save to My Trips, add notes, change status, delete.

Activity Diagram — Plan a Trip



From choosing a destination to a saved trip, including validation and the login detour on Save.

Activity Diagram — Register and Login

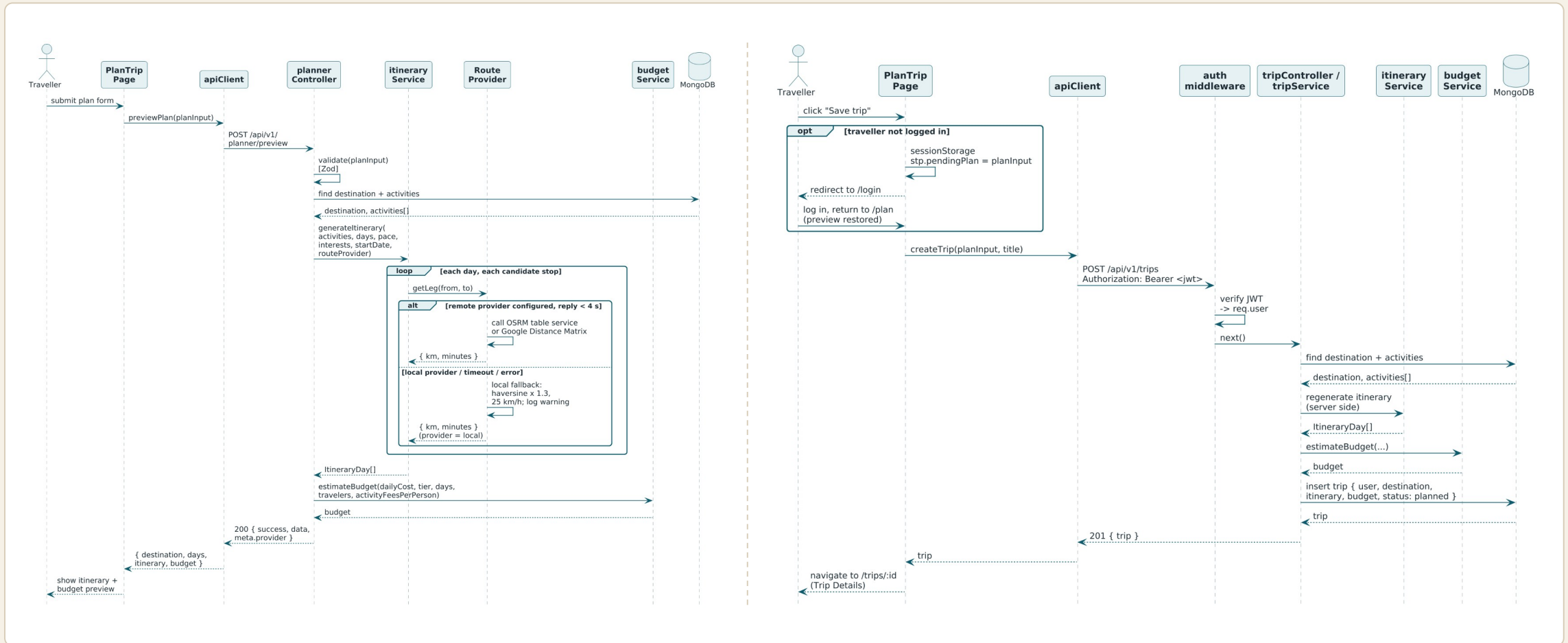


WHAT TO NOTICE

Registration and login share one outcome: a JWT stored on the client and a return to the page the user came from.

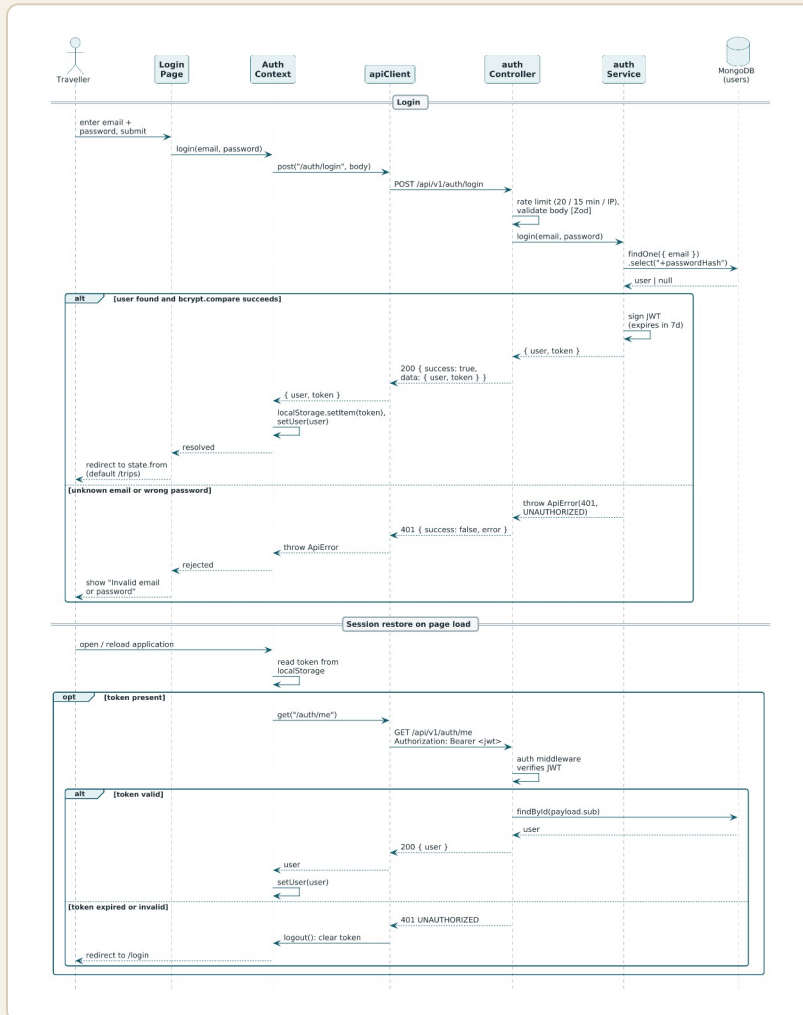
- 1 Duplicate email ends in a conflict message, not a crash.
- 2 Wrong credentials give one generic error.
- 3 After success the user returns to the page that asked for login.

Interaction Diagram — Plan and Save (Sequence)



Preview is public; on Save the server recomputes the itinerary and budget before writing the trip.

Interaction Diagram — Login (Sequence)



WHAT TO NOTICE

Credentials go in once; every later request carries the JWT in the Authorization header.

1

The password hash is read only for comparison and never returned.

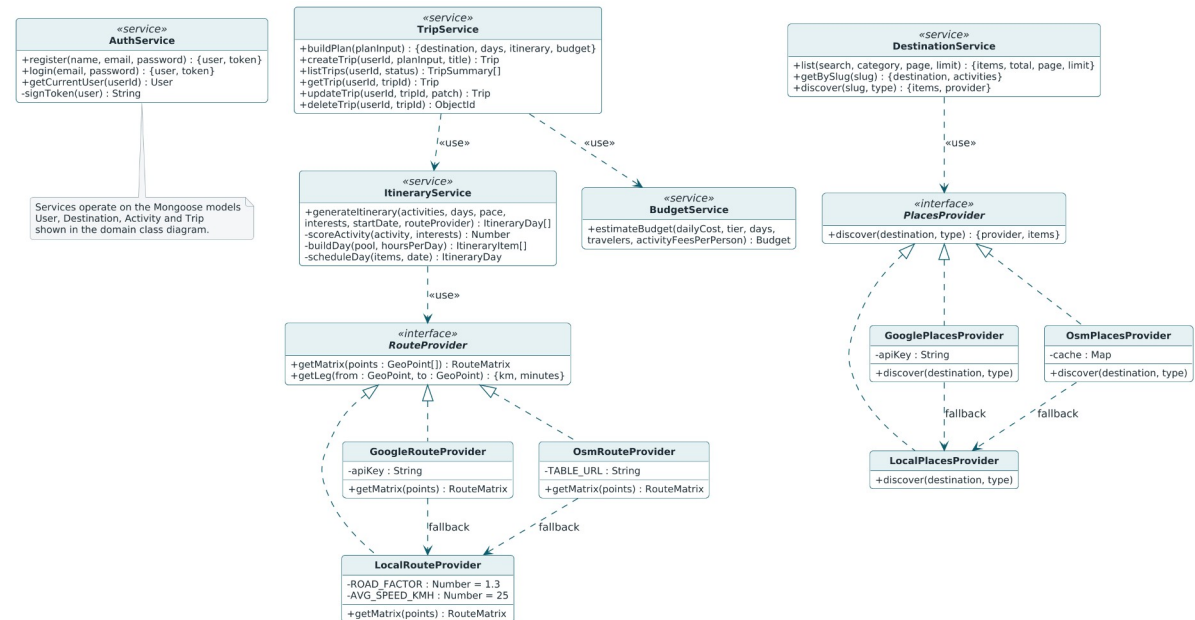
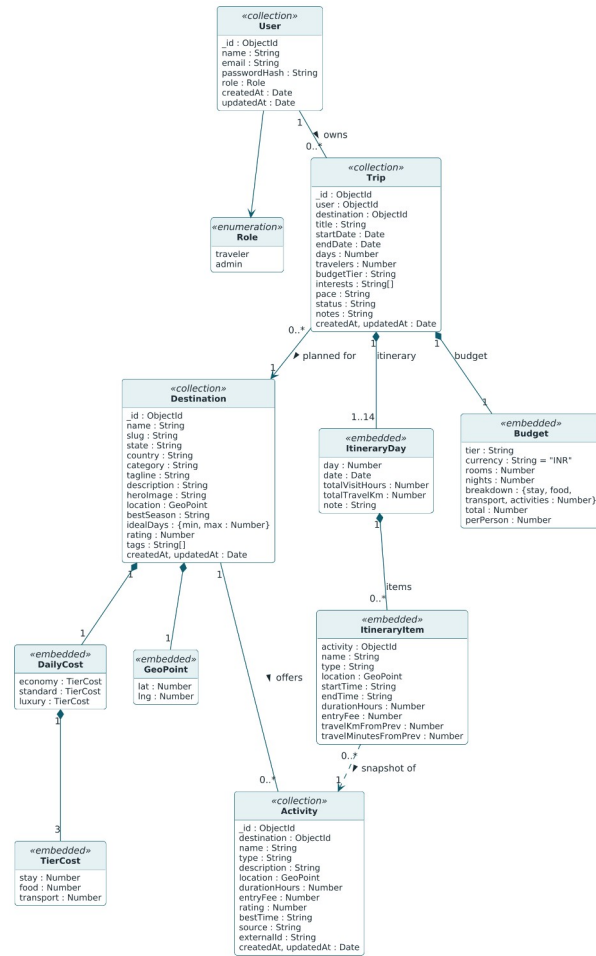
2

The token is kept in localStorage; /auth/me restores the session.

3

A 401 on any later call logs the user out and redirects to login.

Class Diagram



Left: four persistent classes, with a Trip composed of days, items and a budget.
 Above: the service classes and provider interfaces.

- 1 User 1 — many Trip;
Destination 1 — many Activity and Trip.
- 2 ItineraryItem is a snapshot of an Activity, so a saved trip never changes.
- 3 Route and Places providers are interfaces; OpenStreetMap and Google implementations fall back to local.

Data Dictionary — Collections and users

Field name	Datatype	Constrains	Description
name	String	required, 2–60 characters, trimmed	Display name
email	String	required, unique, lowercase, valid email	Login identifier
passwordHash	String	required, select: false	bcrypt hash (cost 10); never sent in JSON
role	String	enum traveler admin, default traveler	Authorisation role

HOW THE FOUR COLLECTIONS RELATE



Common fields

_id: ObjectId, createdAt, updatedAt on every collection.

Money

INR, stored as integer rupees. No decimals anywhere.

Indexes

email unique · slug unique · { destination, name } unique · trips.user indexed.

Data Dictionary — destinations

Field name	Datatype	Constrains	Description
name	String	required	Destination name, e.g. Manali
slug	String	required, unique, lowercase	URL identifier, e.g. manali
state	String	required	Indian state or union territory
country	String	default India	Country
category	String	enum: hill-station, beach, heritage, spiritual, nature, desert, adventure	Primary category, used by the filter
tagline	String	required, ≤ 90 characters	One-line hook shown on cards
description	String	required	Overview of 2–4 sentences
heroImage	String	required	/images/destinations/<slug>.jpg
location	{ lat, lng }	required, Numbers	City-centre coordinates
bestSeason	String	required	e.g. October – February
idealDays	{ min, max }	required, Numbers	Suggested trip length
dailyCost	{ economy, standard, luxury } → { stay, food, transport }	required, integers ≥ 0	stay per room per night; food, transport per person per day
rating	Number	0–5	Editorial rating
tags	[String]	optional	Search keywords

Data Dictionary — activities

Field name	Datatype	Constraints	Description
destination	ObjectId → destinations	required, indexed	Parent destination
name	String	required; unique with destination	e.g. Solang Valley
type	String	enum: sightseeing, adventure, culture, nature, spiritual, food, shopping	Used for interest matching
description	String	required	1–2 sentences
location	{ lat, lng }	required	Real-world coordinates
durationHours	Number	required, 0.5–8	Typical visit time
entryFee	Number	integer ≥ 0, default 0	Per person, INR
rating	Number	0–5	Feeds the activity score
bestTime	String	enum: morning, afternoon, evening, any; default any	Scheduling hint
source	String	enum: seed, google; default seed	Where the record came from
externalId	String	optional	Google place id when source = google

Data Dictionary — trips

Field name	Datatype	Constraints	Description
user	ObjectId → users	required, indexed	Owner of the trip
destination	ObjectId → destinations	required	Planned destination
title	String	required, ≤ 80 characters	Default: "<days> days in <destination>"
startDate	Date	required	First day of the trip
endDate	Date	required, ≥ startDate	Last day of the trip
days	Number	1–14	Inclusive day count
travelers	Number	1–12	Group size
budgetTier	String	enum: economy, standard, luxury	Cost tier used for the estimate
interests	[String]	subset of activity type enum	Boosts matching activities
pace	String	enum: relaxed, balanced, packed; default balanced	Hours of sightseeing per day
itinerary	[ItineraryDay]	embedded	Generated day-wise plan
budget	Budget	embedded	Generated cost estimate
status	String	enum: planned, completed, cancelled; default planned	Trip state
notes	String	≤ 1000 characters, default ""	Traveller's own notes

Data Dictionary — Structures Embedded in a Trip

ItineraryDay

Field	Type	Description
day	Number	Day number, from 1
date	Date	Calendar date
items	[ItineraryItem]	Ordered stops
totalVisitHours	Number	Sum of visit durations
totalTravelKm	Number	Sum of leg distances
note	String	e.g. free-day message

Budget

Field	Type	Description
tier	String	economy standard luxury
currency	String	Always "INR"
rooms, nights	Number	Rooms and nights used for stay
breakdown	Object	stay, food, transport, activities
total	Number	Sum of the breakdown
perPerson	Number	round(total / travelers)

ItineraryItem (snapshot of an activity)

Field	Type	Description
activity	ObjectId	Source activity
name	String	Copied activity name
type	String	Copied activity type
location	{ lat, lng }	Copied coordinates
startTime	String HH:mm	Scheduled start
endTime	String HH:mm	Scheduled end
durationHours	Number	Visit time
entryFee	Number	Per person, INR
travelKmFromPrev	Number	Km from previous stop
travelMinutesFromPrev	Number	Minutes from previous stop

Implementation Status

87%

13 of 15 modules
complete

Main flow works end to
end: browse → plan →
preview → login → save
→ manage.

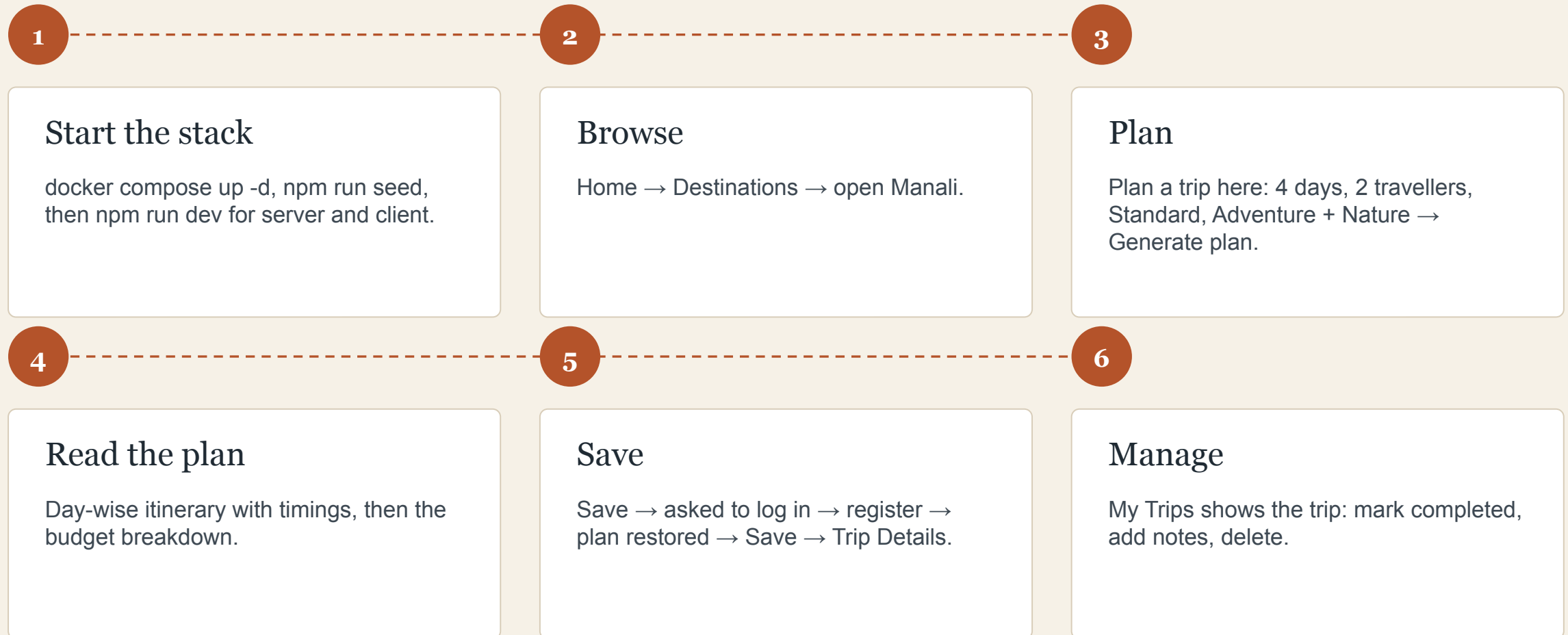
Backend — Express API

Data models + seed (12 destinations, activities)	DONE
Auth API — register, login, me (JWT, bcrypt)	DONE
Destinations API — list, filter, details	DONE
Itinerary service — scoring, routing, scheduling	DONE
Budget service — tier-based estimate	DONE
Trips API — save, list, update, delete, ownership	DONE
OpenStreetMap places + OSRM road distances (live)	DONE
Google Places / Distance providers — live-key testing	IN PROGRESS
Automated tests — unit + API	DONE

Frontend — React client

Home, Destinations, Destination details	DONE
Plan Trip form + generated plan preview	DONE
Login / Register, protected routes, plan restore	DONE
My Trips + Trip Details (notes, status, delete)	DONE
Discover nearby places panel	DONE
Accessibility audit + cross-browser testing	IN PROGRESS

Implementation Status — Demo Flow



Thank you

We would be glad to take your questions and suggestions.

Udit Mishra

Backend / Database / API

Hardik Vaghela

Frontend / UI

Vraj **[[SURNAME]]**

Documentation / UML / System Design

GUIDED BY

[[GUIDE NAME]]